PHASE 4 - ADVANCED | DAY 22 OF 30

Data & Pandas

DataFrames, CSV I/O, groupby(), merge()/join(), and plotting for real-world tabular data

2 hrs	5	20	Intermediate+
SESSION LENGTH	CONCEPT FILES	PRACTICE PROBLEMS	DIFFICULTY

30-Day Python Programming Course • Instructor Session Guide

Session Overview

Today students move from NumPy's raw numerical arrays (Day 21) to Pandas — the library that gives every data professional labelled, tabular superpowers. By the end of this session, students will load real CSV data, summarize it with groupby(), combine multiple tables with merge(), and produce their first charts.

Goal	Give students hands-on fluency with the Pandas DataFrame/Series API for loading, cleaning, summarizing, combining, and visualizing tabular data.
Outcome	Students can read/write CSVs, filter and aggregate data with groupby(), join two related tables, and produce a basic chart from a DataFrame.
Instructor Note	Pandas syntax is dense at first. Lean on the IRCTC/Swiggy/Flipkart-style examples to keep every method grounded in a relatable scenario.

Session Timeline

Time	Segment	Focus
0:00 - 0:15	Warm-Up & Recap	Recap Day 21 NumPy; introduce why Pandas exists
0:15 - 0:35	Step 1	DataFrame & Series basics
0:35 - 0:55	Step 2	Reading & writing CSV files
0:55 - 1:05	Break	5-10 minute breather
1:05 - 1:30	Step 3	groupby() aggregations
1:30 - 1:50	Step 4	merge() and join()
1:50 - 2:10	Step 5	Basic plotting with df.plot()
2:10 - 2:20	Wrap-Up	Recap, key concepts, preview Day 23

Prerequisites & Preparation

Requirement	Details
Python & environment	Python 3.12 with the venv from Day 1 activated
Pandas installed	pip install pandas --break-system-packages (or uv add pandas)
Matplotlib installed	pip install matplotlib --break-system-packages (for Step 5)
Prior knowledge	Days 6 (data structures), 8 (file I/O), 21 (NumPy)
Files ready	day22_01 through day22_05 concept files on every student machine

Instructor Pre-Session Checklist

[ ] Verify pandas and matplotlib import cleanly on the training machine/projector

[ ] Pre-run all 5 concept files once to warm up caches and confirm no errors

[ ] Have the IRCTC/Swiggy/Flipkart example datasets ready to reference on-screen

[ ] Prepare a blank terminal + VS Code with day22_01 open for live coding

[ ] Review common groupby() and merge() confusion points before class (Step 3-4)

Step-by-Step Instruction Guide

STEP 1 - DataFrame & Series Basics (20 min)

"Today we move from NumPy's number grids to Pandas' labelled tables. Think of a DataFrame as an Excel sheet you can fully control with code. Let's build one from scratch using student marks."

```
import pandas as pd

students = pd.DataFrame({
    "name": ["Priya Sharma", "Arjun Mehta", "Riya Deshmukh"],
    "city": ["Mumbai", "Delhi", "Pune"],
    "marks": [88, 76, 92],
})
print(students)
# Output:
#           name    city  marks
# 0  Priya Sharma  Mumbai     88
# 1   Arjun Mehta   Delhi     76
# 2  Riya Deshmukh    Pune     92
```

Walk through **.loc[]** (label-based) vs **.iloc[]** (position-based) selection slowly - this is the #1 source of confusion for beginners. Use the whiteboard analogy: ".loc is like calling someone by name, .iloc is like calling out their seat number."

Error	Cause	Fix
KeyError on column name	Typo or wrong case in column name	Check df.columns for exact spelling
Single [] vs double [[]]	students['name','marks'] fails	Use students[['name','marks']] for multiple columns
IndexError with .iloc	Position beyond the number of rows	Check df.shape[0] before indexing

DONE WHEN: Every student can create a DataFrame from a dict, select a column with df['col'], and filter rows with a boolean condition like df[df['marks'] > 80].

STEP 2 - Reading & Writing CSV Files (20 min)

"Real data almost never starts inside your code - it lives in CSV files, exports from apps like IRCTC or Swiggy. Let's save our DataFrame and load it right back."

```
students.to_csv("students.csv", index=False)

reloaded = pd.read_csv("students.csv")
print(reloaded.dtypes)
# Output:
# name      str
# city      str
# marks    int64
```

Emphasize **index=False** when saving - forgetting it adds an unwanted "Unnamed: 0" column when reading back in. Also demo **dtype={}** for protecting ID-like columns (e.g., PNR numbers) from being silently converted to integers.

Error	Cause	Fix
FileNotFoundError	CSV path wrong or file not yet saved	Confirm working directory with os.getcwd()
Extra 'Unnamed: 0' column	Forgot index=False when saving	Always pass index=False unless index is meaningful
Leading zeros dropped from IDs	Pandas auto-converts to int	Use dtype=str for ID-like columns

DONE WHEN: Every student can save a DataFrame to CSV with to_csv(index=False) and reload it with read_csv(), and can explain why na_values= is useful for messy data.

STEP 3 - groupby() Aggregations (25 min)

"groupby() answers questions like 'average order value per city'. Remember the three-step mental model: split the data into groups, apply a function to each group, then combine the results back together."

```
avg_by_city = swiggy.groupby('city')['order_value'].mean()
print(avg_by_city)
# Output:
# city
# Bengaluru    373.33
# Delhi        386.67
# Mumbai       537.50
# Name: order_value, dtype: float64
```

Show **.agg(name=("col","func"))** for multiple named aggregations at once - this is the pattern students will reuse constantly in the capstone project (Day 28 analytics layer).

Error	Cause	Fix
Nothing prints after groupby()	Forgot to apply an aggregation function	groupby() alone is lazy; chain .mean()/.sum()/etc.
MultiIndex confusion	Grouped by multiple columns	Use .reset_index() to flatten back to columns
Wrong column aggregated	Selected column before or after groupby differs	Double-check df.groupby('x')['y'] targets the right column

DONE WHEN: Every student can group a DataFrame by one or more columns, apply mean/sum/count, and use .agg() to compute multiple named summary statistics at once.

STEP 4 - merge() and join() (20 min)

"Data lives in separate tables, just like SQL. Customers in one table, orders in another. merge() is how we bring them together - and the join TYPE decides what happens to rows that don't have a match."

```
result = pd.merge(customers, orders, on='customer_id', how='left')
print(result)
# Output (partial):
#   customer_id          name  order_id  amount
# 0         101  Priya Sharma      ORD1  1200.0
# ...
# 4         104  Sanjay Verma       NaN     NaN   <- no orders
```

Draw a simple Venn diagram on the whiteboard for inner/left/right/outer - this visual sticks far better than the code alone. Relate it back to SQL JOINs if any students have database experience.

Error	Cause	Fix
MergeError: no common columns	Join column names differ between tables	Use left_on= and right_on= explicitly
Duplicate rows after merge	One-to-many relationship (expected!)	Confirm with .groupby() if duplication is intended
Unexpected NaN values	Used outer/left/right instead of inner	Choose join type based on which rows you must keep

DONE WHEN: Every student can explain the difference between inner/left/right/outer joins and can merge two related DataFrames on a shared key column.

STEP 5 - Basic Plotting with df.plot() (20 min)

"Numbers in a table are hard to feel. A chart tells the story instantly. Pandas gives us .plot() for free - it's Matplotlib underneath, just with a shortcut."

```
import matplotlib.pyplot as plt

avg_by_city.plot(kind='bar', title='Avg Order Value by City')
plt.tight_layout()
plt.savefig('avg_order_value.png')
# Output:
# A bar chart image saved as avg_order_value.png
```

On lab machines without a display, remind students to call **matplotlib.use('Agg')** before importing pyplot, and to always call **plt.savefig()** followed by **plt.close()** so charts don't pile up in memory.

Error	Cause	Fix
No display / backend error	Running on a headless machine	matplotlib.use('Agg') before importing pyplot
Chart looks empty/blank	Forgot to select the right column/Series	Confirm the Series has non-zero values before plotting
Overlapping x-axis labels	Too many categories, default rotation	plt.xticks(rotation=45) or use kind='barh'

DONE WHEN: Every student can produce and save a line, bar, and pie chart from a DataFrame or Series using df.plot(), with a title and axis labels.

Session Wrap-Up

Student Checklist

[ ] Can create a DataFrame from a dict, list of dicts, or NumPy array

[ ] Can select rows/columns using .loc[], .iloc[], and boolean filtering

[ ] Can read and write CSV files with pd.read_csv() / .to_csv()

[ ] Can group data with groupby() and compute mean/sum/count/.agg()

[ ] Can merge two DataFrames using inner/left/right/outer joins

[ ] Can produce a basic line, bar, or pie chart with df.plot()

Key Concepts Recap

Concept	One-Line Summary
DataFrame / Series	Labelled 2-D table / 1-D column, built on top of NumPy
.loc[] / .iloc[]	Select rows by label vs. by integer position
read_csv() / to_csv()	Load and save tabular data from/to disk
groupby()	Split-apply-combine pattern for summarizing data by category
merge() / join()	Combine two related tables on a shared key, SQL-style
df.plot()	Quick charts (line/bar/pie/hist) built on Matplotlib

Next Day Preview

Day 23 - Web Requests & APIs: Students will learn the requests library (GET/POST, headers, params), core REST API concepts, JSON response parsing, authentication headers, and the async httpx client.

Troubleshooting Notes

If a student's pandas import fails, confirm they're in the correct virtual environment (Day 1 setup) and that pandas was installed with `pip install pandas --break-system-packages` or `uv add pandas`. Chained assignment warnings (SettingWithCopyWarning) are common when filtering then modifying - recommend `.copy()` after filtering to avoid this.

Quick Reference Card

Daily Workflow Commands

```
# Activate environment (from Day 1)
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate         # Windows

# Run today's files
python day22_01_dataframe_series_basics.py
python day22_practise_problems_01_dataframe_series.py
```

Pandas Cheat Sheet

Task	Code
Create DataFrame	`pd.DataFrame({'col': [...]})`
Read CSV	`pd.read_csv('file.csv')`
Write CSV	`df.to_csv('file.csv', index=False)`
Select column(s)	`df['col'] / df[['col1','col2']]`
Select rows by label	`df.loc[row_label]`
Select rows by position	`df.iloc[row_position]`
Filter rows	`df[df['col'] > value]`
Group and aggregate	`df.groupby('col')['target'].mean()`
Multiple aggregations	`df.groupby('col').agg(name=('c','func'))`
Merge two tables	`pd.merge(df1, df2, on='key', how='inner')`
Plot a chart	`df.plot(kind='bar', title='...')`

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + Enter	Run current cell/selection in interactive window
Shift + Enter	Run current line and move to next
Ctrl/Cmd + `	Toggle integrated terminal
F5	Run and debug the current file

Tool Summary

pandas - DataFrame/Series, CSV I/O, groupby, merge | **matplotlib** - underlying plotting engine for df.plot() | **numpy** - powers Pandas' numerical operations under the hood